

Protocol Parsing for Exploit Development and Offensive Engineering (Sections 1–6)

1. What is Protocol Parsing?

Protocol parsing is the process of transforming raw bytes into structured meaning.

Mental Model:

bytes → tokens → structure → meaning

Examples of protocols include:

- HTTP
 - Redis RESP
 - SMB
 - DNS
 - Custom malware C2 protocols
 - File formats such as PNG, PDF, and PE files
-

2. Why Parsers Matter in Exploit Development

Exploit developers target parsers constantly because parsers process attacker-controlled input.

Parsers are responsible for:

- Reading lengths
- Allocating memory
- Processing binary data
- Handling recursion
- Managing arrays and buffers

This makes parsers common attack surfaces for:

- Buffer overflows
 - Heap corruption
 - Integer overflows
 - Use-after-free vulnerabilities
 - Out-of-bounds reads/writes
-

3. Framing Messages

Framing answers:

'Where does one message begin and end?'

Common framing methods:

1. Delimiter-Based

Example:
PING\r\n

2. Length-Prefixed
Example:
[length][data]

3. Fixed-Size Structures
Example:
struct Header {
 uint32_t cmd;
 uint32_t size;
};

4. Serialization and Deserialization

Serialization:
object → bytes

Deserialization:
bytes → object

Example packet:

```
struct Packet {  
  uint32_t command;  
  uint32_t length;  
  char data[];  
};
```

Understanding serialization is critical for:

- Malware communication
 - Beacon tasking systems
 - C2 infrastructure
 - Reverse engineering protocols
-

5. State Machines

Most protocol parsers are implemented as state machines.

Example states:

- WAITING_FOR_HEADER
- READING_LENGTH
- READING_DATA
- PROCESS_COMPLETE

Example pseudo-code:

```
switch(state) {
```

```
case READ_HEADER:
...
break;

case READ_DATA:
...
break;
}
```

6. Critical Security Concepts

Parser security depends on strict validation.

Never trust:

- Length fields
- Packet sizes
- User input
- Claimed offsets

Common dangerous mistakes:

1. Buffer Overflow

```
char buf[256];
recv(sock, buf, attacker_length, 0);
```

2. Integer Overflow

```
int total = len + header;
```

3. Out-of-Bounds Access

```
buf[index];
```

4. Heap Corruption

```
malloc(attacker_size);
```

Attackers frequently manipulate malformed packets to trigger these vulnerabilities.
